0015. 3Sum

ty: Medium | Category: Two Pointers | LeetCode: https://leetcode.com/problems/

Problem

- Return all unique triplets [a, b, c] from nums with a + b + c == 0. No duplicate triplets.
- Example: [-1,0,1,2,-1,-4] → [[-1,-1,2],[-1,0,1]].

Intuition

- Sort. Fix the first element nums[i]; the rest is Two Sum II on the suffix with target -nums[i].
- Sorting also makes duplicates adjacent, so skipping equal neighbours removes duplicate triplets.

Brute Force

- Three nested loops plus a set of sorted triplets: O(n^3).

Optimal Approach

- Sort nums.
- For i in 0..n-3: skip if i > 0 and nums[i] == nums[i-1]; break if nums[i] > 0.
- L = i+1, r = n-1. While l < r: sum < 0 → l++; sum > 0 → r--; sum == 0 → record, l++, then skip duplicates of nums[l].

Worked Example

```
sorted = [-4,-1,-1,0,1,2].

i=0 (-4): l/r never reach 0 -> none
i=1 (-1): l=2(-1) r=5(2) sum=0 -> [-1,-1,2]; l=3
          l=3(0)  r=5(2) sum=1 -> r=4
          l=3(0)  r=4(1) sum=0 -> [-1,0,1]; l=4 stop
i=2 (-1): duplicate of i=1 -> skip
i=3 (0) : l=4 r=5 sum=3 -> r-- stop
```

Java Solution

```java
public class ThreeSum {
    public List<List<Integer>> threeSum(int[] nums) {
        Arrays.sort(nums);
        List<List<Integer>> res = new ArrayList<>();
        for (int i = 0; i < nums.length - 2; i++) {
            if (nums[i] > 0) break;
            if (i > 0 && nums[i] == nums[i - 1]) continue;
            int l = i + 1, r = nums.length - 1;
            while (l < r) {
                int sum = nums[i] + nums[l] + nums[r];
                if (sum < 0) l++;
                else if (sum > 0) r--;
                else {
                    res.add(List.of(nums[i], nums[l], nums[r]));
                    l++;
                    while (l < r && nums[l] == nums[l - 1]) l++;
                }
            }
        }
        return res;
    }
}
```

Complexity

- Time: O(n^2). Space: O(1) besides output (sort may use O(log n)).

Pitfalls

- Skip duplicates for both i and l (after a match), or you emit repeats.
- Early break when nums[i] > 0 is an optimisation, not required.